

Agentic Screening Feature

Custom Python orchestration for HR resume screening

What this feature does

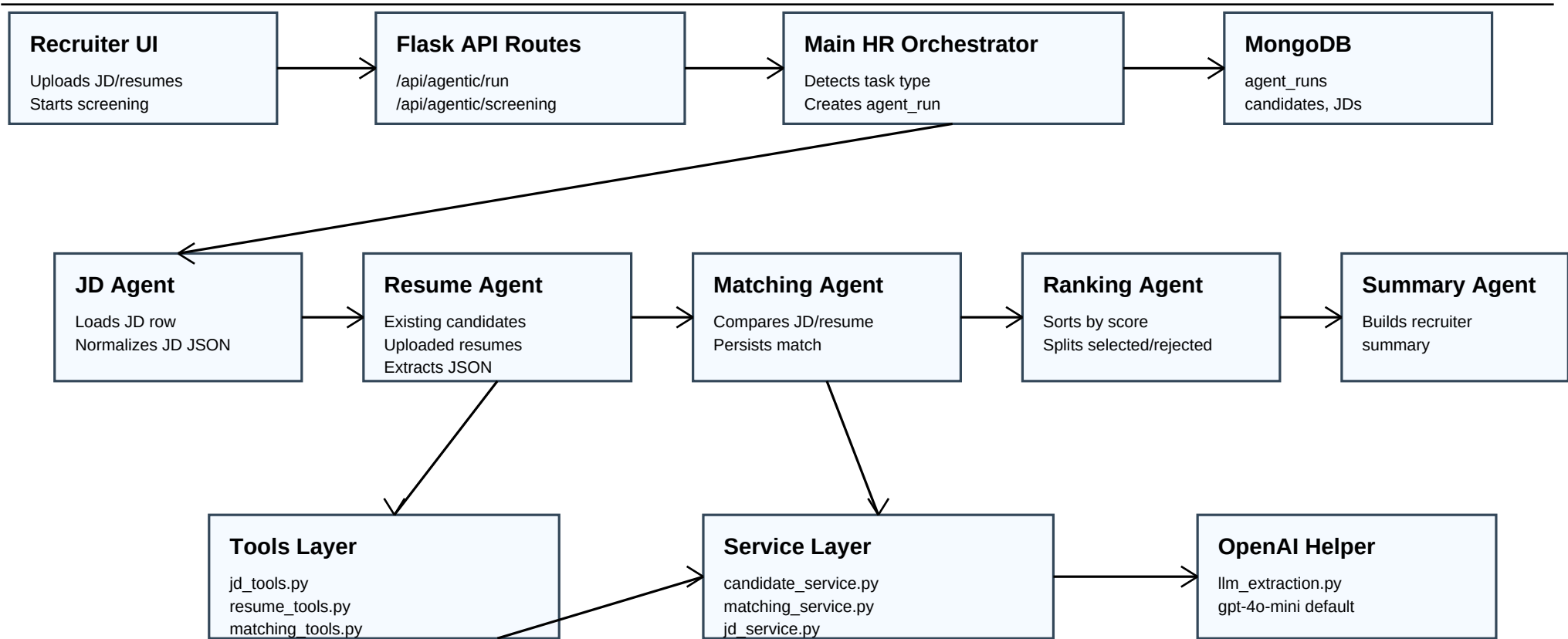
- Recruiter submits a JD and resumes or existing candidate IDs through Flask APIs.
- Main HR Orchestrator creates a run record and routes the task to the correct workflow.
- Screening Workflow runs specialized agents in sequence: JD, Resume, Matching, Ranking, Summary.
- OpenAI is used through the local `llm_extraction` helper where extraction or scoring needs LLM support.
- LangChain and LangGraph are listed as dependencies, but this code path is a custom Python pipeline.

Key modules

- `routes/agentic_routes.py`: API layer for `/api/agentic/run` and `/api/agentic/screening`.
- `orchestration/main_orchestrator.py`: creates run IDs, stores status, and dispatches task types.
- `workflows/screening_workflow.py`: sequential screening pipeline.
- `agents/*.py`: role-specific Python agents.
- `tools/*.py` and `services/*.py`: reusable database, extraction, matching, and persistence logic.

Architecture Diagram

Runtime structure of the custom agent pipeline

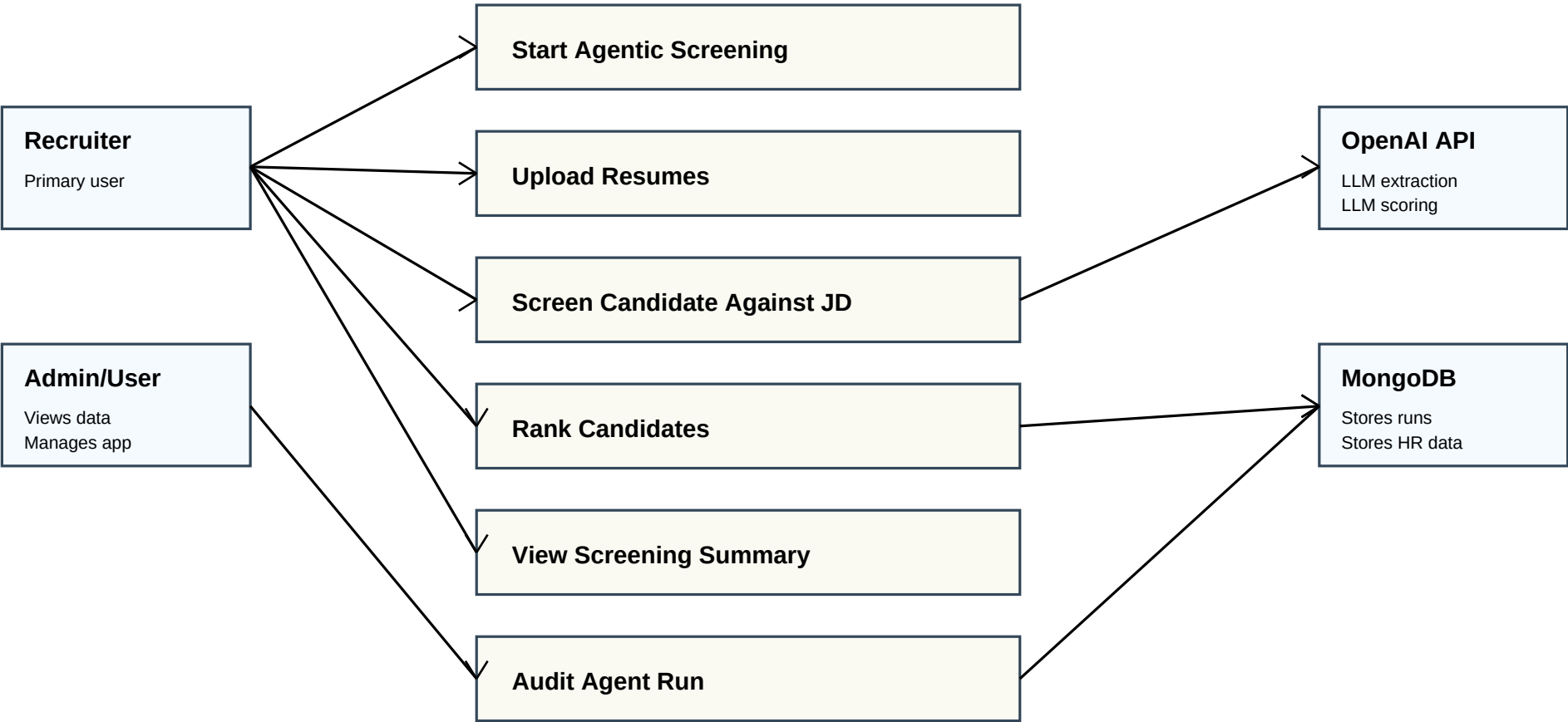


Note: This architecture is agentic by design, but it is not implemented with LangGraph nodes or LangChain runnables.

The agents are normal Python functions with tracing and shared helper tools.

Use Case Diagram

Main actors and system responsibilities

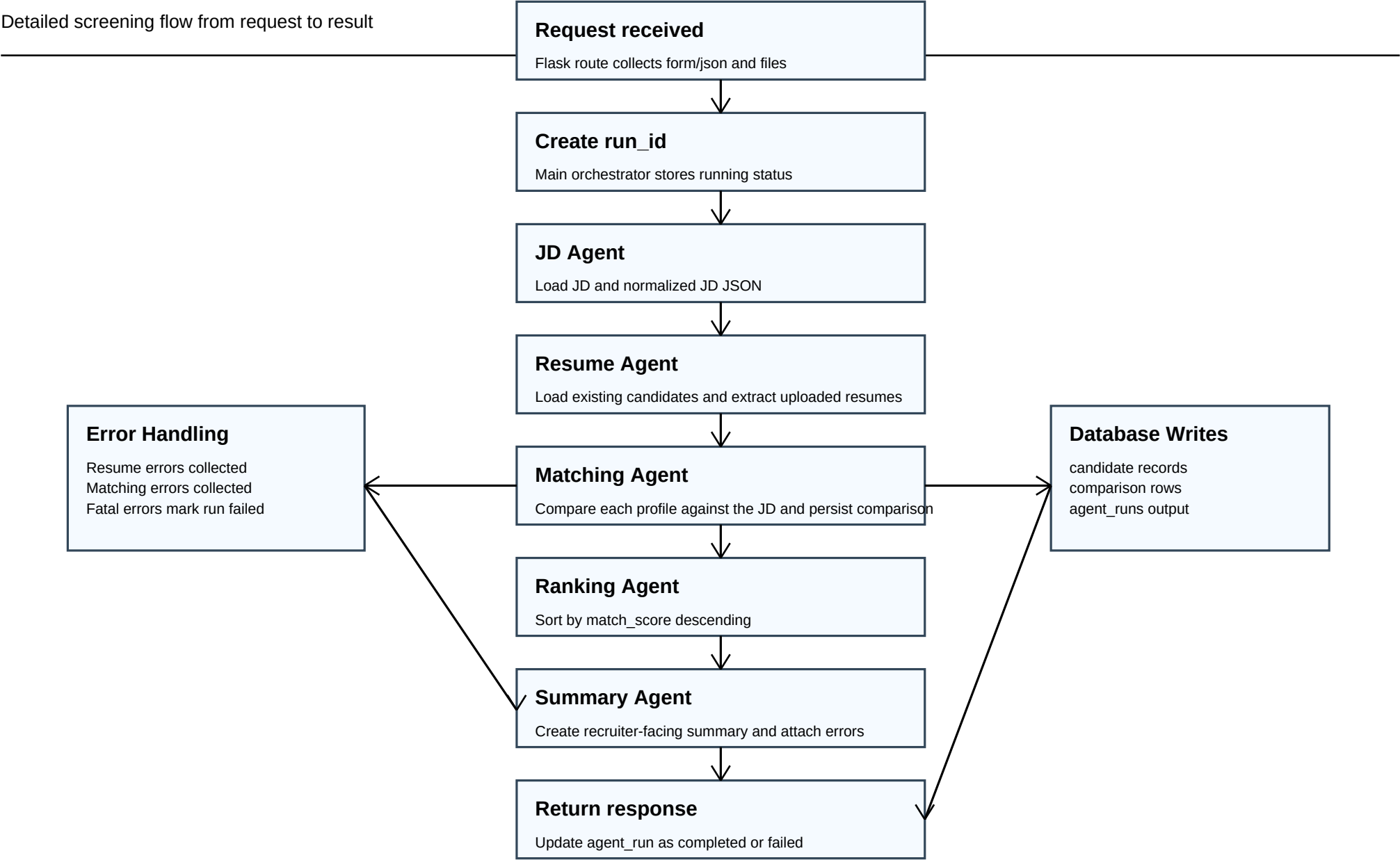


Recruiter goal: quickly identify selected, rejected, and top-ranked candidates for a JD.

System goal: keep every run traceable with run_id, task_type, input, output, status, and errors.

Flow Chart Diagram

Detailed screening flow from request to result



Ranking detail: the current Ranking Agent uses deterministic sorting by match_score, not an LLM call.